

Code Explanation

Data Processing Pipeline Overview

The provided Python code is a data processing pipeline built using Apache Spark and Delta Lake. It reads customer and order data from CSV files, cleans and processes the data, and then saves it to Delta tables. The pipeline also creates an order summary table with SCD Type 2 implementation and a customer aggregate spend table.

Step 1: Creating a Spark Session

This step creates a SparkSession object, which is the entry point to any Spark functionality.

```
def create_spark_session() -> SparkSession:
    return SparkSession.builder
        .appName("Customer Order Processing")
        .getOrCreate()
```

Step 2: Reading Source Data

This step reads customer and order data from CSV files into DataFrames.

```
def read_source_data(spark: SparkSession) -> Tuple[DataFrame, DataFrame]:
    customer_df = spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customer
data")

    order_df = spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdat
a")

    return customer_df, order_df
```

Step 3: Cleaning Data

This step cleans the customer and order data by removing nulls and duplicates.

```
def clean_data(customer_df: DataFrame, order_df: DataFrame) -> Tuple
[DataFrame, DataFrame]:
    customer_df = customer_df.na.drop()
        .dropDuplicates(["CustId"])

    order_df = order_df.na.drop()
        .dropDuplicates(["OrderId"])

    order_df = order_df.withColumn("TotalAmount", col("PricePerUnit"
) * col("Qty"))

    return customer_df, order_df
```

Step 4: Saving Data to Delta Tables

This step saves the cleaned customer and order data to Delta tables.

```
def save_to_delta(df: DataFrame, catalog: str, schema: str, table: str) -> None:
    df.write
        .format("delta")
        .mode("overwrite")
        .option("overwriteSchema", "true")
        .saveAsTable(f"{catalog}.{schema}.{table}")
```

Step 5: Creating Order Summary Table

This step creates an order summary table with SCD Type 2 implementation.

```
def create_order_summary_table(spark: SparkSession, customer_df: DataFrame, order_df: DataFrame) -> None:
    joined_df = customer_df.join(order_df, "CustId", "inner")
    .select(
        "CustId",
        "Name",
        "EmailId",
        "Region",
        "OrderId",
        "ItemName",
        "PricePerUnit",
        "Qty",
        "Date"
    )

    if not table_exists:
        scd_df = joined_df
        .withColumn("IsActive", lit(True))
        .withColumn("StartDate", current_timestamp())
        .withColumn("EndDate", lit(None).cast(TimestampType()))

        save_to_delta(scd_df, "gen_ai_poc_databricksco", "sdlc_wizard", "ordersummary")
    else:
        update_scd_type2_table(spark, joined_df)
```

Step 6: Updating SCD Type 2 Table

This step updates the SCD Type 2 table with new data.

```
def update_scd_type2_table(spark: SparkSession, new_data: DataFrame) -> None:
    delta_table = DeltaTable.forName(spark, "gen_ai_poc_databricksco.sdlc_wizard.ordersummary")

    new_data = new_data
        .withColumn("IsActive", lit(True))
        .withColumn("StartDate", current_timestamp())
```

```

        .withColumn("EndDate", lit(None).cast(TimestampType()))
delta_table.alias("target").merge(
    new_data.alias("source"),
    """
    target.CustId = source.CustId AND
    target.OrderId = source.OrderId AND
    target.IsActive = true AND
    (
        target.Name != source.Name OR
        target.EmailId != source.EmailId OR
        target.Region != source.Region OR
        target.ItemName != source.ItemName OR
        target.PricePerUnit != source.PricePerUnit OR
        target.Qty != source.Qty OR
        target.Date != source.Date
    )
    """
)
    .whenMatchedUpdate(
        set={
            "IsActive": lit(False),
            "EndDate": current_timestamp()
        }
    )
    .whenNotMatchedInsertAll()
    .execute()

```

Step 7: Creating Customer Aggregate Spend Table

This step creates a customer aggregate spend table.

```

def create_customer_aggregate_spend(spark: SparkSession) -> None:
    order_summary_df = spark.table("gen_ai_poc_databrickscoe.sdlc_wizard.ordersummary")

    if "TotalAmount" not in order_summary_df.columns:
        order_summary_df = order_summary_df.withColumn("TotalAmount",
            col("PricePerUnit") * col("Qty"))

    aggregated_df = order_summary_df
        .filter(col("IsActive") == True)
        .groupBy("Name", "Date")
        .agg(spark_sum("TotalAmount").alias("TotalAmount"))

    aggregated_df.write
        .format("delta")
        .mode("overwrite")
        .saveAsTable("gen_ai_poc_databrickscoe.sdlc_wizard.customera
ggregatespend")

```

Step 8: Main Execution

This step executes the data processing pipeline.

```
def main():
    spark = create_spark_session()

    customer_df, order_df = read_source_data(spark)
    customer_df, order_df = clean_data(customer_df, order_df)

    save_to_delta(customer_df, "gen_ai_poc_databrickscoe", "sdlc_wizard", "customer")
    save_to_delta(order_df, "gen_ai_poc_databrickscoe", "sdlc_wizard", "order")

    create_order_summary_table(spark, customer_df, order_df)
    create_customer_aggregate_spend(spark)

if __name__ == "__main__":
    main()
```